

## Interview Questions — Java - Spring Boot

Total Questions: 40

---

1. What is Spring Boot and how is it different from traditional Spring Core?

**Hint:** Think about configuration, setup time, and embedded server support.

---

2. Why is auto-configuration considered a benefit in Spring Boot applications?

**Hint:** Consider how much manual configuration is needed in regular Spring.

---

3. What is the role of the embedded server (like Tomcat) in a Spring Boot application?
- 

4. How would you create a new Spring Boot project using Spring Initializr?

**Hint:** Focus on the steps and options you select during project creation.

---

5. Explain the difference between GET, POST, PUT, and DELETE HTTP methods in the context of REST APIs.

**Hint:** Think about what each method is typically used for.

---

6. What does the `@RestController` annotation do in a Spring Boot application?
- 

7. How would you map a URL path to a specific method in your controller using Spring Boot annotations?

**Hint:** Consider which annotation is used for mapping endpoints.

---

8. In a REST API, how do you extract a value from the URL path (for example, `/events/5`) inside a controller method?

**Hint:** Think about which annotation helps bind URL segments to method parameters.

---

9. Write a simple method in a Spring Boot controller that accepts a JSON object in the request body and prints one of its fields.

**Hint:** Remember which annotation is used to bind the request body to a method parameter.

---

10. What is Spring Data JPA and how does it simplify database operations?

**Hint:** Focus on how it reduces the amount of code needed for CRUD operations.

---

- 
11. What is the purpose of the Entity, Repository, and Service layers in a typical Spring Boot application?
- 
12. How does the CrudRepository interface help you perform database operations in Spring Boot?
- 
13. Suppose you need to save an Event object to a MySQL database using Spring Data JPA. Briefly describe the steps required to achieve this.
- 
14. How would you define a simple JPA entity class for an Event with fields: id, name, and date?
- Hint:** Consider which annotations are needed to mark a class and its fields as a JPA entity.
- 
15. What is Spring Security and why is it important in web applications?
- Hint:** Think about authentication and protecting resources.
- 
16. How does Basic Authentication work in a Spring Boot REST API?
- Hint:** Consider what information is sent with each request and how the server verifies it.
- 
17. What does the @Valid annotation do when applied to a method parameter in a controller?
- Hint:** Think about input validation and error handling.
- 
18. Give an example of how you would use @NotNull in a Spring Boot entity class.
- 
19. If you want to protect the 'Add Event' API so only authenticated users can access it, what changes would you make in your Spring Boot project?
- Hint:** Think about configuration and annotations related to security.
- 
20. How would you test a REST API endpoint created in Spring Boot using Postman?
- 
21. What is CORS and why do you need to enable it when connecting a React frontend to your Spring Boot backend?
- Hint:** Consider how browsers handle requests from different origins.
- 
22. Suppose your React frontend fails to fetch data from the /events API due to a CORS error. What would you check and how would you fix it in your Spring Boot application?
- 
23. Describe the steps to verify that your /events API is returning the correct data using Postman.
-

24. How would you structure your backend code to separate concerns between controllers, services, and repositories in a Spring Boot project?

**Hint:** Think about what each layer is responsible for.

25. In your mini project, how would you implement user registration and login with security in Spring Boot?

26. Explain how you would perform CRUD operations for event management in your Campus Connect Portal backend.

27. How would you integrate your Spring Boot REST API with a React frontend using Axios?

28. What does it mean for your backend to be 'microservice-ready'? How would you design your Spring Boot application to support this?

**Hint:** Consider scalability, modularity, and separation of concerns.

29. What are some important steps to prepare your Spring Boot application for deployment on platforms like Heroku or Render?

30. How would you use Postman's AI features to automate API test case generation for your project?

**Hint:** Think about how AI can suggest or generate test cases based on your API schema.

31. Suppose you want to automatically test all CRUD endpoints of your /events API. How can Postman collections and AI help streamline this process?

32. How could you use ChatGPT or GitHub Copilot to help you debug a problem in your Spring Boot application, and what steps would you take to ensure the suggested solution is reliable?

**Hint:** Consider code review, testing, and understanding the AI's suggestions.

33. If you use an AI tool to generate boilerplate code for a new REST controller in Spring Boot, what should you always verify before using the code in your project?

34. Integrative: How would you secure the POST /events endpoint using both Basic Authentication and input validation? Briefly outline the annotations and configuration you would use.

**Hint:** Think about combining security and validation features.

35. Integrative: Describe the flow of a request from a React frontend to a Spring Boot backend that fetches a list of events, mentioning CORS, the controller, service, repository, and database.

**36.** Integrative: If a user is unable to add a new event due to a validation error, but is authenticated, how would you troubleshoot and resolve the issue in your Spring Boot application?

**Hint:** Consider both security and validation aspects.

**37.** Write a Spring Boot controller method that updates an event's name using its ID, and explain how you would test this endpoint using Postman.

**38.** Suppose your deployed Spring Boot application on Heroku is not connecting to the MySQL database. What steps would you take to diagnose and fix this issue?

**39.** How does using Service and Repository layers make your Spring Boot application more maintainable and testable?

**Hint:** Think about separation of business logic and data access.

**40.** What are some common mistakes freshers make when designing REST APIs in Spring Boot, and how would you avoid them?